SITE RELIABILITY ENGINEERING • OUTAGE POST-MORTEM

Google Maps Global Outage Dissection: Bad Rollouts & Cascading Retries

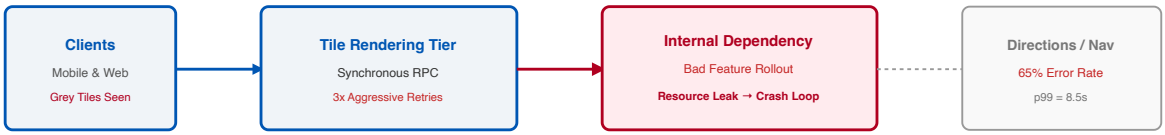
Dissection of Map Tiles API Failure, Synchronous Dependency Deadlocks, and Circuit Breaker Design

EXECUTIVE SUMMARY

On March 18, 2022, a bad feature deployment in an internal shared service triggered a resource exhaustion crash loop. Because Google's core Map Tile rendering engine had an unbuffered synchronous RPC dependency on this service, failing requests spawned immediate $3 imes$ retries. The resulting $4 imes$ traffic surge overwhelmed in-memory queues, crashing the tile rendering tier globally and rendering blank grey tiles worldwide for over 2.5 hours.

1. INCIDENT TOPOLOGY: SYNCHRONOUS TILE RENDERING

Google Maps separates raster and vector tile generation into microservices. The frontend tile server communicates synchronously with downstream data enrichment engines:



2. ROOT CAUSE: RESOURCE EXHAUSTION & RETRY MULTIPLICATION

The incident was triggered by a code release that contained a subtle memory/thread leak. Upon receiving production traffic, every daemon instance exhausted its allocated heap within seconds, crashing repeatedly.

Downstream Crash Loop

Instances rebooted, initialized, accepted inbound RPCs, immediately exhausted resources, and crashed again. Availability dropped to 0%, resulting in 100% RPC timeouts on callers.

Upstream Queue Explosion

The Tile Rendering tier did not implement circuit breaking. Instead, it executed 3 immediate retries per failed RPC, multiplying system traffic by $4 imes$ while waiting 2.0s per timeout.

3. MATHEMATICAL FORMALISM: LITTLE'S LAW QUEUE EXPANSION

The collapse of the tile rendering tier is explained by Little's Law ($L = \lambda W$), where L is queue depth, λ is request arrival rate, and W is service duration:

```
Healthy State:
  \lambda_{normal} = 100,000 req/sec,  W_{normal} = 20 ms (0.02s)
  In-Flight Queue: L_{normal} = 100,000 * 0.02 = 2,000 requests in memory

Failed State (100% downstream failure + 3 retries):
  \lambda_{amplified} = 100,000 * (1 + 3) = 400,000 req/sec
  W_{timeout} = 2,000 ms (2.0s)
  In-Flight Queue: L_{critical} = 400,000 * 2.0 = 800,000 requests in memory!

Buffer Growth Factor: L_{critical} / L_{normal} = 400x memory demand!
Result: Instant process heap exhaustion (OOM) and JVM/Go runtime GC thrashing.
```

4. OUTAGE INCIDENT CHRONOLOGY

Time	Component	Observed Telemetry & Anomaly	Severity
T + 00:00	Internal Service	New feature binary deployed to production clusters globally.	Rollout Canary skipped
T + 00:05	Internal Service	Process memory leaks; daemons enter crash-reboot loops.	Crash 0% Availability
T + 00:08	Tile Renderer	Synchronous RPC timeouts trigger $4 imes$ retry storm.	4x Traffic Saturation
T + 00:15	Client Apps	Users see blank grey tiles globally; SDK error rate hits 65%.	Critical Sitewide Outage
T + 00:45	SRE Team	Bad deployment identified and rolled back to previous stable build.	Rollback Reverted
T + 02:30	Global Platform	Client backlogs and TCP socket queues finally drain; metrics normalize.	Resolved Normal State

5. DEEP ARCHITECTURAL LESSONS

- **Hard Synchronous Coupling:** Tile rendering should never fail completely due to a secondary data enrichment service. Missing metadata should render base tiles gracefully.
- **The Recovery Hysteresis:** Rolling back a bad deployment took 35 minutes, but the platform required nearly 2 hours to recover due to client retry backlogs and cold cache penalties.

6. ARCHITECTURE SOLUTIONS: GOOGLE'S REMEDIATION STRATEGY

Google implemented four strategic hardening practices across its frontend and microservice architectures:

1. Early Request Rejection

Drop inbound requests immediately (HTTP 429 / 503) when server CPU exceeds 85%, preventing memory buffer bloat.

2. Queue Bounding

Enforce strict caps on in-memory buffers and TCP backlogs so unserviceable requests fail fast at the network socket.

3. Priority Circuit Breakers

Automatically shed internal analytics, batch syncs, and telemetry traffic to preserve 100% capacity for user navigation.

7. RESILIENT CIRCUIT BREAKER IMPLEMENTATION WITH FULL JITTER

```
// Production Circuit Breaker with Decorrelated Jitter Backoff
func FetchTileWithCircuitBreaker(ctx context.Context, cb *CircuitBreaker, req *TileRequest) (*TileData, error) {
    if !cb.Allow() {
        return DefaultPlaceholderTile(req), nil // Graceful degradation!
    }

    baseDelay := 50 * time.Millisecond
    for attempt := 0; attempt < 3; attempt++ {
        tile, err := rpcCall(ctx, req)
        if err == nil {
            cb.RecordSuccess()
            return tile, nil
        }
        cb.RecordFailure()

        // Full Jitter Formula: rand(0, base * 2^attempt)
        jitter := time.Duration(rand.Int63n(int64(baseDelay * (1 << attempt))))
        time.Sleep(jitter)
    }
    return DefaultPlaceholderTile(req), nil
}
```

8. PRODUCTION ENGINEERING CHECKLIST

Resilience Invariants for Large-Scale Distributed Microservices

1. **Enforce Graceful Degradation:** If a secondary microservice fails, synthesize fallback responses rather than failing the core request.
2. **Bound In-Memory Queues:** Never allow request queues to grow without strict upper limits; reject excess requests fast.
3. **Cap Retries with Budgets:** Enforce client retry budgets (maximum 10% retry traffic) to avoid cascading traffic amplification.
4. **Apply Full Jitter on Retries:** Always introduce uniform randomness into backoff calculations to avoid synchronized waves.
5. **Prioritize Customer Traffic:** Equip load balancers with priority shedding rules to drop internal batch workloads during degradation.